

MPLADS-Sentinel

Dashboard Design Specification

Role-based views, feature rationale, and data provenance for the Ministry, District Authority, and MP dashboards

Problem Statement ID: 26102 — Development of an AI-powered system to detect anomalies, fraud, and inefficiencies in MPLAD Scheme implementation
Organization: Ministry of Statistics and Programme Implementation (MoSPI)
Department: Data Informatics and Innovation Division (DIID)
Category: Software | Theme: Smart Automation
Prepared by: Nidhish (Frontend)

1. Problem Statement Alignment

The official problem statement calls for decision-support dashboards for four stakeholder groups: Members of Parliament, State Nodal Authorities, District Authorities, and the Ministry. This specification covers three of the four — Ministry, District Authority, and MP — as the finalized build scope. The State Nodal Authority view is deliberately descope for the current build phase and noted here as a planned extension, since a state-level rollup would reuse the same shell with a multi-district, single-state filter.

Every feature in this document traces back to a specific line in the problem statement — anomaly detection in expenditure and cost estimates, duplicate and delayed work identification, risk-based alerts, predictive insight, and automated compliance monitoring — and each view below is scoped to what that stakeholder actually needs to act on, rather than a single undifferentiated dashboard.

2. Shared Layout Shell

All three views are built from one shared component shell: a top bar (branding, search, role switch), a left sidebar (role-specific navigation), a KPI summary row, a two-panel mid-section, and a bottom table or detail panel. Only the sidebar items, KPI labels, and mid/bottom panel content change per role — the layout, spacing, and component structure stay identical. This keeps the frontend build to one shell plus three data/permission configurations, rather than three independent pages.

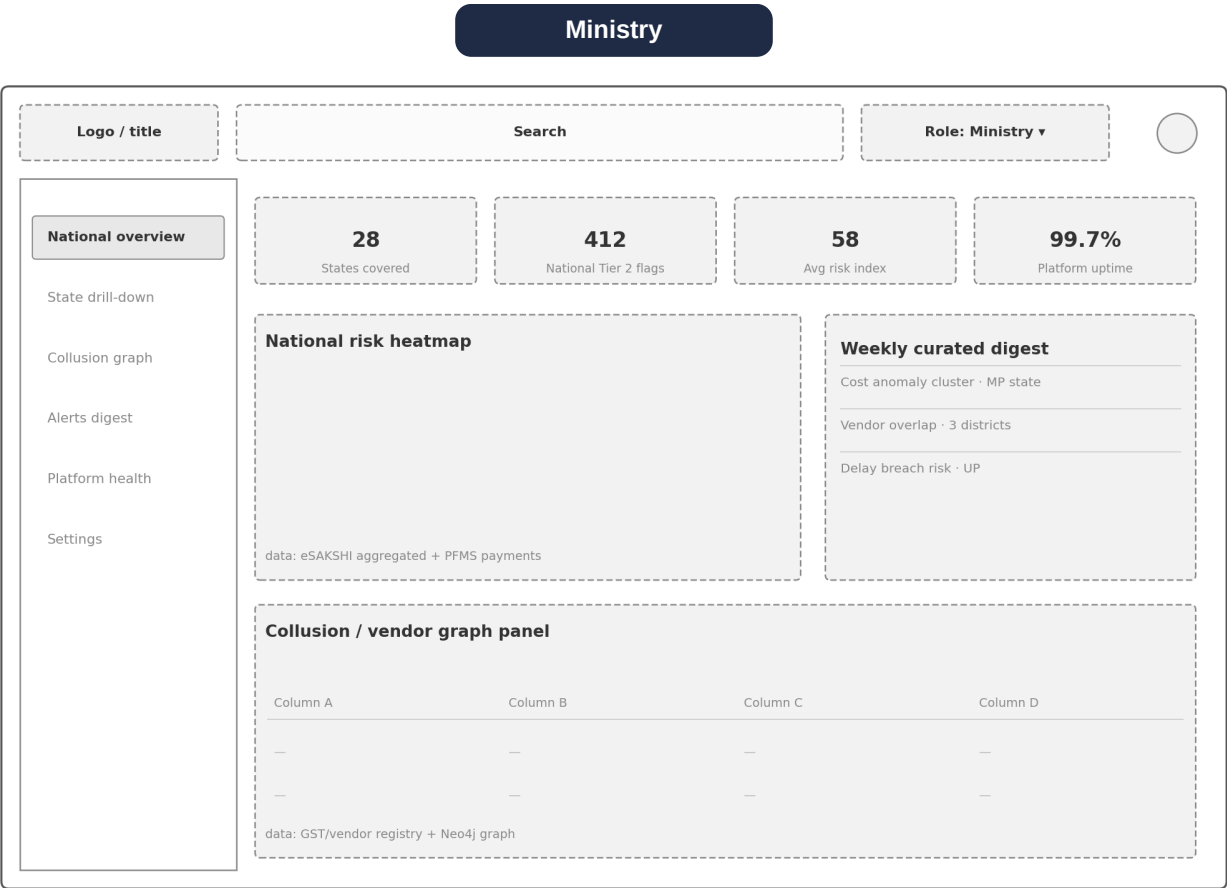


Figure 1: Ministry dashboard — low-fidelity layout

The Ministry view is the widest lens and the most aggregated. It answers a different question than District or MP: not “is this one project okay” but “is there a pattern across states, districts, or contractors that no single lower-level authority could see on its own.” Nothing here is per-project detail — it is aggregated so a Ministry official is not drowning in the same volume of data a District officer would see.

Feature	Data source	Why it's needed
National risk heatmap	eSAKSHI records aggregated across states, cross-referenced with PFMS payment data	Ministry has no single-district lens by default; this is the only place a nationwide pattern (e.g. one state consistently over-flagged) becomes visible at all.
Weekly curated digest	Output of the two-tier alert engine — Tier 2 (corroborated) flags only, volume-capped	A raw alert firehose gets ignored over time (alert fatigue); pre-filtering to corroborated, capped-volume flags is what keeps this channel trusted.
Collusion / vendor graph	GST and vendor registry data, modeled in a Neo4j knowledge graph (MP → District → Agency → Contractor → Work → Payment)	Repeat-winner contractors and shared bank accounts only become visible when viewed across districts; no single District Authority has the cross-jurisdiction visibility to catch this pattern.
Alert precision audit	Human-reviewed outcomes (confirmed vs. dismissed), tracked per alert type over time	A false positive here can wrongly implicate an MP or District Authority — a politically and legally consequential error. Tracking precision explicitly prevents trust erosion in the system.
Platform self-monitoring	Prometheus / Grafana metrics on the platform's own ingestion pipeline	Indian government portals have historically failed under load and eroded public trust; Ministry needs visibility into system health, not just scheme health.
RAG query assistant	LangChain-based retrieval over MPLADS guideline documents and past investigation outcomes	Converts an opaque risk score into a guideline-grounded, plain-language answer — what makes a flag usable as evidence rather than just a suspicion.

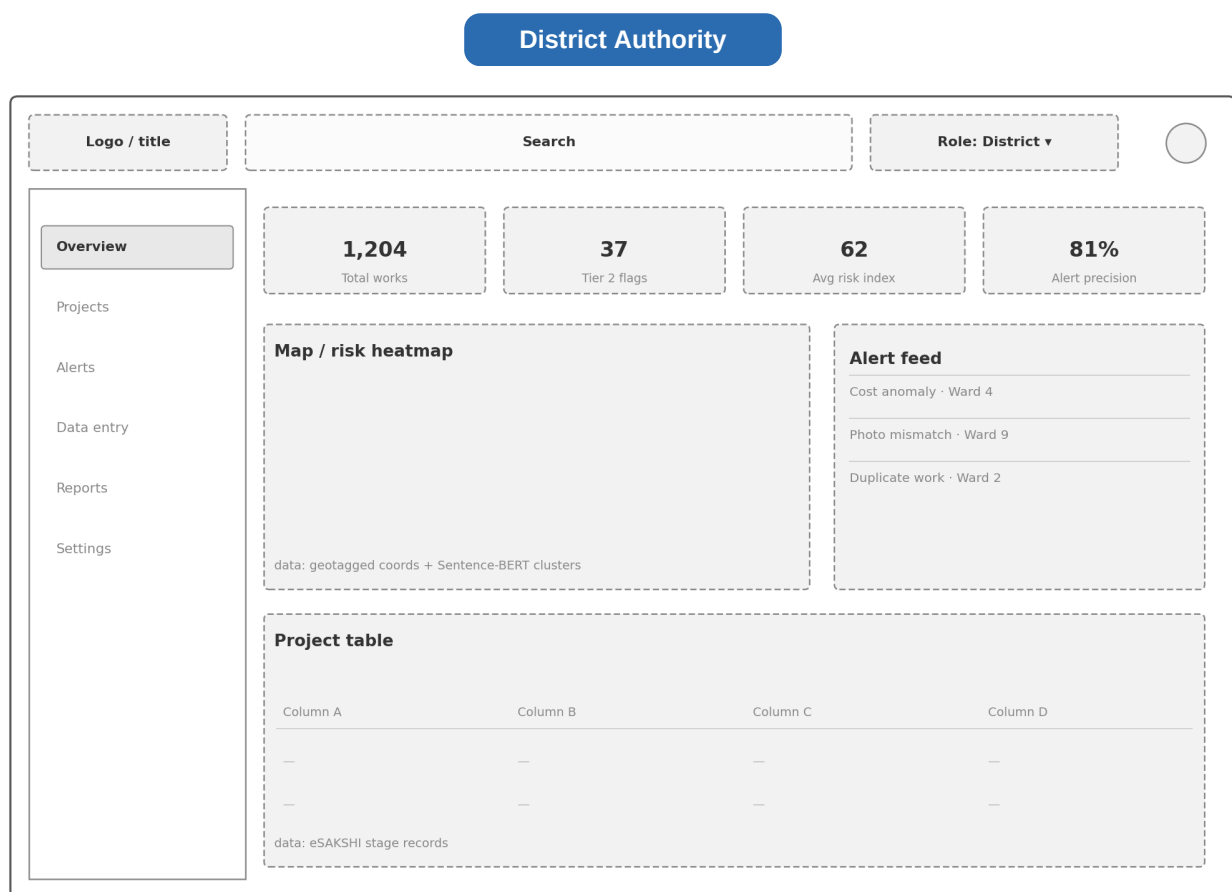


Figure 2: District Authority dashboard — low-fidelity layout

The District Authority view is where the operational detection work happens. This is the role that approves cost estimates, receives progress updates, and is closest to the point where a duplicate work, an inflated cost estimate, or a fabricated completion claim can still be caught before money moves further down the chain.

Feature	Data source	Why it's needed
Jurisdiction project list	eSAKSHI stage records (recommendation → sanction → progress → payment → completion), filtered to the district	This is the operational home screen — every other panel in this view drills down from a row in this table.
Duplicate / overlap map	Sentence-BERT embeddings of work descriptions, clustered geospatially (DBSCAN) on lat/long	Catches the same work sanctioned twice under different names — the highest-value intervention point, since it stops the money before it moves.
Cost anomaly flags	Per-work cost vs. category/region benchmark (Isolation Forest / Z-score)	District Authority is the role that actually approves cost estimates; this is the earliest point an inflated estimate can be stopped.
Photo vs. progress verification	Geotagged stage photos compared against self-reported completion percentage (CNN / Vision Transformer)	Completion status is self-reported by Implementing Agencies — a known integrity gap; this check catches a mismatch between claim and visual evidence.
Completion-lag tracker	Payment status cross-referenced against completion-marking status	Flags works that are fully paid but never marked complete — invisible in a normal status view since nothing looks broken until two fields are compared.

Feature	Data source	Why it's needed
Regional-language data entry	BHASHINI translation API, applied to the data-entry form itself	District and Implementing Agency staff enter data at block/panchayat level, often not English/Hindi-first; a form they cannot read reliably produces the bad data that causes false positives downstream.

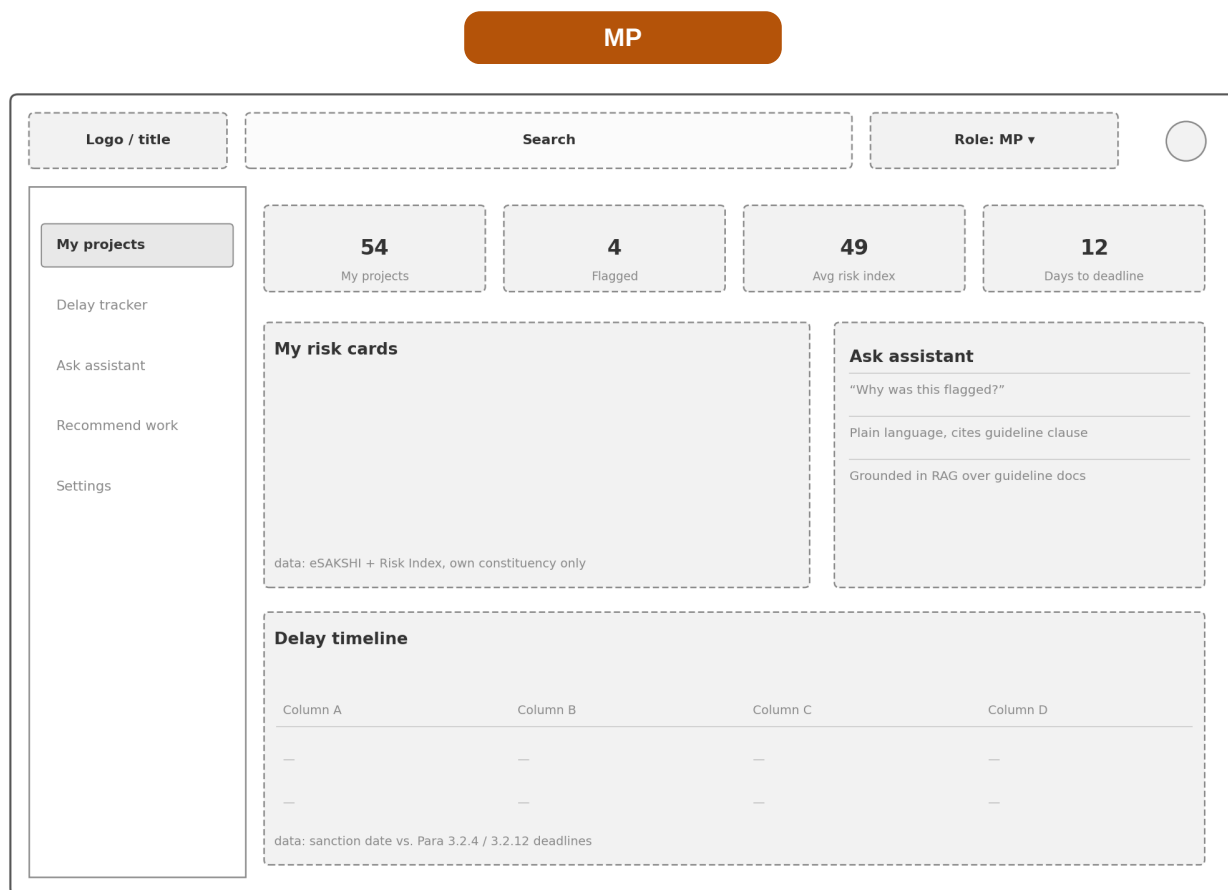


Figure 3: MP dashboard — low-fidelity layout

The MP view is intentionally the thinnest of the three. An MP should see only their own recommended works, scored and explained in plain language — never a cross-district pattern or another constituency's flag, which is exactly the false-positive-as-political-liability scenario the alerting design is built to avoid.

Feature	Data source	Why it's needed
My risk cards	Composite 0–100 Risk Index, filtered to the MP's own recommended works	An MP should never see another constituency's flags; scoping strictly to their own works avoids implicating them in a pattern that isn't theirs.
Delay early-warning	Sanction date checked against the Para 3.2.4 (45-day sanction) and Para 3.2.12 (stipulated completion) guideline timelines, with Prophet-based prediction	Lets an MP intervene before a guideline breach occurs, not just get notified after the fact — the “early” in early-warning.
Completion-lag view	Same payment-vs-completion cross-check as the District view, scoped to the MP's own works	An MP remains accountable for works they recommended even after District takes over execution, so this visibility matters at their level too.
Ask assistant (RAG)	Same guideline and investigation-history retrieval as the Ministry assistant, scoped to the MP's own project context	An MP is unlikely to read a SHAP plot; a plain-language, guideline-grounded answer is what turns a flag into something they can act on or contest.
Recommend work	New-work submission form, feeding directly into the eSAKSHI-mirrored pipeline	Keeps the loop closed — an MP does not just monitor, they also originate new works through the same system that monitors them.

3. Technology Stack

3.1 Frontend

Technology	Purpose
React (Vite) + Tailwind CSS	Core framework and styling for the shared dashboard shell
React Router, role-based route guards	Ministry / District / MP as permission-gated routes on one shell
Recharts	KPI trend charts and the trend-analysis view
Leaflet / Mapbox GL JS	Risk heatmap and geotagged project map
react-force-graph (d3-force)	Collusion / vendor relationship graph
TanStack Table	Sortable, filterable project tables
TanStack Query	Caching and background refetch for frequently updated alerts and risk scores
Zustand	Role and session state
Dexie.js (IndexedDB)	Offline-first data entry with sync-on-reconnect for District field staff
react-i18next + BHASHINI API	Regional-language UI for the data-entry form

3.2 Backend / ML (integration context for the frontend)

Technology	Purpose
Kafka / Apache NiFi	Partitioned ingestion from eSAKSHI, PFMS, GST/vendor registries
Delta Lake (Parquet) + Neo4j	Curated data storage and the relationship graph
Isolation Forest, Sentence-BERT, DBSCAN	Cost anomaly and duplicate/overlap detection
Prophet, GraphSAGE (phased)	Delay prediction and collusion detection as history accumulates
CNN / Vision Transformer	Photo-vs-progress verification
SHAP / LIME	Explainability feeding the risk cards and the RAG assistant
LangChain + vector DB	RAG assistant over guideline documents and investigation history
Prometheus / Grafana	Platform self-monitoring

For build purposes, the frontend does not need to wait on the live ML pipeline: a mock API returning the same response shape as the Risk Index and SHAP output allows the dashboard shell and all three role views to be developed and demoed independently of ML pipeline readiness.